
tneq_qc: Tensor Network Engine toward Quantum: *Yet Another Software But More Flexiable, Learnable, and Distributable*

Yifeng Wang¹ Chao Li^{2,3} Zhun Sun⁴

Abstract

Quantum tensor networks offer a compact language for representing quantum states, circuit-like operators, and high-dimensional learning models, but existing tools often separate quantum-oriented modeling, differentiable training, and scalable execution. We present `tneq_qc`, a flexible software framework that unifies these workflows through a readable ASCII graph interface, modular contraction engines, scaled tensor abstractions, PyTorch and JAX backends, and distributed execution support. The framework enables users to specify quantum tensor-network structures concisely, switch contraction strategies and numerical backends without rewriting models, and train tensor-network parameters end-to-end with automatic differentiation. We illustrate its use in tensor factorization, quantum tensor-network equation matching, and quadratic-form density estimation, and we evaluate distributed training on large tensor-network workloads. The results show that `tneq_qc` can accelerate favorable workloads while reducing memory usage in larger settings, making it a practical platform for quantum-inspired machine learning and scalable tensor-network research. Code is available at <https://github.com/minogame/tneq-qc>.

a large tensor into a network of smaller cores, TNs reveal algebraic structure that can often be exploited for both computational and memory efficiency. Among them, quantum tensor networks (QTNs) are recently paid attention because *not only* they naturally describe quantum states and quantum circuits, and therefore form a direct bridge between tensor methods and quantum computing. Their strong expressivity and close connection to quantum computation have *also* made them increasingly attractive in learning tasks where compact representations and physically meaningful inductive biases are desirable.

Despite this promise, current software ecosystems do not fully support the workflow needed in this setting. Machine-learning-oriented frameworks usually provide mature automatic differentiation and hardware acceleration, but they are rarely designed for quantum-circuit specification or tensor-network contraction tailored to quantum applications. In contrast, physics-oriented tensor-network libraries are often strong in simulation and numerical modeling, yet they typically pay less attention to learning pipelines, especially back-propagation through contractions. In addition, tensor-network computation is intrinsically expensive, so practical applications increasingly require *distributed execution* rather than only efficient single-device implementations. Although excellent toolboxes exist across MATLAB, Python, and C++, it remains difficult to find a framework that combines quantum-oriented modeling, differentiable execution, and distributed tensor-network computation in a unified way.

1. Introduction

Tensor networks (TNs) provide a unified and expressive framework for representing structured high-dimensional objects, and they have linked a broad range of topics including tensor factorization, deep neural network modeling, probabilistic circuits, and scientific computing. By decomposing

Contribution. To address this gap, we develop `tneq_qc`, a compact and extensible framework for QTNs. The library allows users to instantiate various QTN models through a human-readable ASCII-art graph description, and execute contractions through a modular engine that cleanly separates contraction strategy from backend implementation. The current framework supports differentiable execution with PyTorch and JAX, and it also includes a distributed execution stack that partitions tensor-network cores across workers and combines partial contractions through hierarchical reductions. In this way, `tneq_qc` provides a practical research platform for QTN computation in machine learning and computation scenarios.

¹Department of System Information Sciences, Graduate School of Information Sciences, Tohoku University, Sendai, Japan
²RIKEN-AIP ³Graduate School of Health Data Science, Juntendo University, Tokyo, Japan ⁴Center for Language AI Research, Tohoku University, Sendai, Japan. Correspondence to: Chao Li <chao.li@riken.jp>, Zhun Sun <sun.zhun.a5@tohoku.ac.jp>.

2. Background on Quantum Tensor Networks

We briefly review basic concepts related to QTNs, which are the main focus in `tneq_qc`.

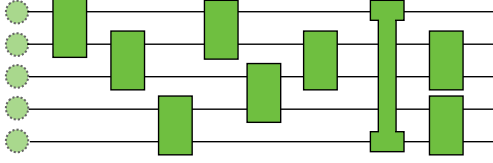


Figure 1. Graphical representation of a quantum tensor network. Nodes denote tensors, edges denote modes, and edges connecting two nodes indicate contracted modes. In this illustration, circular nodes represent vector-like tensors, while square nodes represent operator-like tensors.

Tensor. A tensor is a multi-dimensional array of real or complex numbers. Scalars, vectors, and matrices correspond to tensors of *order* zero, one, and two, respectively, and higher-order tensors extend these notions to additional modes (i.e., the array directions). In this paper, the order of a tensor refers to the number of modes, while the size along each mode specifies its dimension.

Contraction. Tensor contraction is the basic computational primitive underlying tensor-network methods. Given two tensors, contraction selects one or more modes of equal dimension and sums over them, producing a new tensor whose remaining modes are inherited from the inputs. Matrix multiplication is a standard special case of tensor contraction. Since nearly all model evaluations in `tneq_qc` reduce to sequences of contractions, friendly-use and numerically stable contraction is central to the framework.

Graphical representation. Tensor networks are naturally represented as graphs, where each node denotes a tensor and each edge denotes a mode (see Figure 1 for example). An edge connecting two nodes indicates a contracted mode, while a dangling edge indicates an open mode that remains in the output. This graphical notation is useful because it exposes both model structure and contraction dependencies in a compact form.

Quantum tensor networks. In this work, we focus on QTNs, namely tensor networks with circuit-like topologies aligned to qubit lines. As illustrated in Figure 1, these networks organize tensors along parallel “qubit axes” and are particularly well suited to describing quantum states, circuit circuit, and more general linear operators. This structured topology is the main target of the graph abstractions and execution strategies implemented in `tneq_qc`.

3. Software Architecture

`tneq_qc` adopts a layered architecture that explicitly decouples *model specification*, *contraction strategy*, and *nu-*

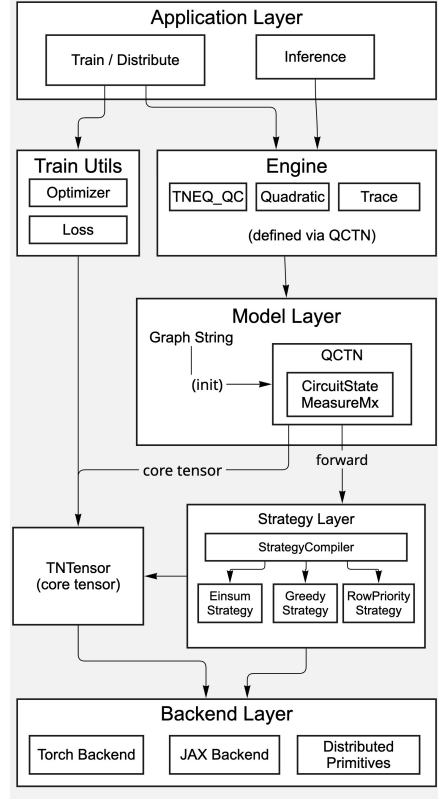


Figure 2. Software architecture of `tneq_qc`.

merical execution, enabling flexible and reusable tensor-network workflows (Figure 2). A key design choice is that tensor networks are defined once via a human-readable ASCII graph, and can be executed under different strategies, optimization routines, and hardware backends without modifying the model.

The system is centered around a unified **engine abstraction** that orchestrates contraction, differentiation, and specialized operations (e.g., trace and quadratic forms) over QTNs. This abstraction isolates execution semantics from implementation details, allowing consistent forward and backward computation across settings.

The **model layer** encodes QTNs as composable graph structures (the `QCTN` class), while the **strategy layer** compiles these graphs into optimized contraction plans using interchangeable methods (e.g., `einsum`, `greedy`). This *strategy-execution separation* enables independent development and evaluation of contraction algorithms.

At the lowest level, the **backend layer** provides interchangeable numerical implementations (e.g., PyTorch, JAX, distributed primitives), supporting automatic differentiation and scalable execution. A unified tensor wrapper (`TNTensor`) ensures consistent handling of scaling, gradients, and metadata across backends.

4. Core Interfaces and Usage

In this section, we illustrate the basic function of `tneq_qc`, from modeling QTN instances to building a workflow for learning tasks using QTNs.

4.1. Define a QTN instance with the `QCTN` class.

A QTN instance can be created directly from an ASCII-art graph specification together with a selected numerical backend. The following example shows a minimal setup:

```
from tneq_qc import QCTN, BackendFactory

backend = BackendFactory.create_backend("pytorch", device="cpu")

graph = """\
-3-A-----3-
-3-A--3--B-----3-
-3-----B--3--C-----3-
-3-----C--3--D-3-
-3-----D-3-
"""

qtn = QCTN(graph, backend).auto_init()
```

As shown above, the structure of a QTN is defined by an ASCII-art multiline string, in which each line corresponds to one “qubit”, the repeated symbols in the same letter represent tensors, and digits “over” the lines indicate the mode dimensions. The helper `QCTNHelper` provides generators for standard structures (e.g., MPS, tree, and brick-wall-like patterns) that can be used for experiments and benchmarks. The `auto_init()` method initializes in random the values of core tensors for the QTN.

4.2. Basic operations on QTNs.

Once we instantiated a QTN, the software supports various QTN operations. For example,

```
qtn_hermit = qtn.hermit()
```

returns a new QTN, whose structure and core tensors are conjugate transposed.

In addition, we can use two or three lines to implement contraction over a QTN as follows:

```
from tneq_qc import EngineCommon
engine = EngineCommon(backend)
t_full = engine.contract(qtn)
```

Note that `engine` models the specific contraction strategy. Using the contraction engine, we can further implement the partial trace for a QTN:

```
#trace out all dangling modes
qtn_f = qtn.clone()
qtn_f.set_trace("all")
tf = engine.contract(qtn_f)
```

```
#trace out the first two "qubits"
qtn_p = qtn.clone()
qtn_p.set_trace([0,1])
tp = engine.contract(qtn_p)
```

The returned `tf` and `tp` are `TNTensor` objects, which is used to store underlying tensors. `TNTensor` supports automatic rescaling via `auto_scale()`, Hermitian views through `hermit()` or `conj_transpose()`, standard arithmetic operations with scale propagation, direct access to gradient-related attributes such as `requires_grad` and `grad`, and batch-aware computation through the flag `has_batch=True`.

Suppose we initiate two QTNs, then we can merge their structure together, returning a new QTN:

```
q1 = QCTN(graph, backend).auto_init()
q2 = QCTN(graph, backend).auto_init()
qtn_concat = QCTN.concat([q1, q2])
```

On the other side, a QTN can be also split into two QTNs:

```
split_idx=3
qlf, qrf = qtn_concat.chunk(index)
```

In the code above, `split_idx=3` core tensors are assigned with the QTN `qlf`, and the remainings are assigned with `qrf`. Note that the `concat` supports the input with more than two QTNs, but `chunk` only support splitting into two QTNs in the current version.

4.3. Backend configuration: PyTorch and JAX.

`BackendFactory` instantiates a backend. Backends implement a common interface for tensor creation/conversion, einsum execution, JIT hooks, and optimizer updates. This enables running the same model code on different numerical stacks and devices while keeping the contraction logic unchanged.

4.4. Distributed deployment

The distributed interface follows the same design principle as the standalone engine: users first define a QTN, then choose an execution backend and a contraction engine. To enable distributed execution, one additionally specifies a communication backend and a partition rule that assigns QTN cores to multiple workers. The engine then performs local contractions on each worker and combines intermediate results through distributed communication. In this way, the same model can be executed in either standalone or distributed mode with only minimal changes in the application code.

A minimal usage example is shown below:

```
from tneq_qc.distributed import
```

```
DistributedTrainer, DistributedConfig

config = DistributedConfig(
    backend_type='pytorch',
    device='cpu',
    qctn_graph=graph,
    comm_backend='torch',
    use_distributed=True,
    rank=rank,
    world_size=world_size,
    partition_strategy='layer',
)

trainer = DistributedTrainer(config)
loss = trainer.engine.train_step(
    circuit_states_list=circuit_states_list,
    measure_input_list=measure_input_list,
    optimizer=optimizer,
    measure_is_matrix=True,
)
```

In the code, `DistributedConfig` specifies both execution settings and distributed metadata, such as the worker rank, world size, and partition strategy. `DistributedTrainer` initializes the communication module, the distributed contraction engine, and the local QTN partition. The call `train_step(...)` executes one distributed forward-backward pass, where each worker contracts its local subnetwork and synchronizes boundary information through the communication backend. This interface provides a simple entry point for distributed training and can also be used as the basis for distributed inference.

4.5. Workflow for QTN learning.

A key feature of `tneq_qc` is that it supports end-to-end *back-propagation* through tensor-network contractions. This makes the framework suitable not only for scientific computing, but also for quantum-inspired machine learning tasks in which the core tensors are treated as trainable parameters. A typical learning workflow can be summarized in five steps:

1. **Prepare data and model.** Encode the input data in tensor-network form and instantiate the trainable QTN model.
2. **Choose backend and engine.** Select a numerical backend such as PyTorch or JAX, and pair it with a contraction engine that determines how the network is executed.
3. **Define the objective.** Construct the loss from the contraction output, for example through likelihood, regression, or classification objectives.
4. **Optimize core tensors.** Use built-in gradient-based

optimization to update the tensor cores until convergence.

5. **Reproducibility.** Use the `save_cores` methods to save check points in the learning process.

5. Examples and Numerical Evaluation

In this section, we use typical TN applications, including tensor factorization and probability density estimation, to illustrate how we use few lines of codes with the proposed software to touch the goal. After that, we use a numerical experiment to demonstrate the speed up for tensor computation using the software’s distributed function.

The repository includes runnable examples under `codes/examples/` and unit tests under `codes/tests/`. These serve both as documentation and as regression checks for contraction strategies, distributed primitives, and probability/sampling utilities.

5.1. Tensor factorization

In this experiment, we model QTNs with the software to approximate images from the MNIST () dataset.

Data preparation. We random draw a batch of images (`BATCH_SIZE=4`) from MNIST, and then resize each image to 32×32 with rescaling the values in the range $[0, 1]$. Finally, images are reshaped into tensors of order 10 with mode dimensions equaling 2. Note that, in the code, we need to define the data with `TNTensor` to support the batch mode.

```
# data in torch.tensor [B, 2, ..., 2]
target = TNTensor(data, has_batch=True)
```

QTN preparation. As introduced in aforementioned sections, QTNs can be easily defined with the software. In the experiment, we use

```
graph = QCTNHelper.mps(NUM_LINES, BOND_DIM, DIM_MODE)
# graph = QCTNHelper.brickwall(NUM_LINES, NUM_LAYERS, DIM_MODE)
qtn = QCTN(graph, backend=backend)
```

Once instantiating the QTN, we need additional steps:

```
for c in qtn.cores:
    core = qtn.cores_weights[c]
    core.requires_grad_(True)
```

The codes allows users to customize which core tensors should be optimized in the learning process, flexiably handling different practical tasks.

The learning process is as follows:

```
opt = create_optimizer("adam", qtn.parameters(), backend=backend, lr=LR)
```

```
for step in range(1, NUM_EPOCHS + 1):
    loss_val, grads = engine.
    contract_for_gradient(qtn, target=t_mnist,
    loss='mse')
    opt.step(list(grads))
```

In the code, `create_optimizer` defines the optimizer, e.g. ADAM, and `engine.contract_for_gradient` handles the loss function, modeling the approximation from QTN to target with various metrics, e.g., MSE.

Numerical results. Figure 3 illustrates the numerical re-

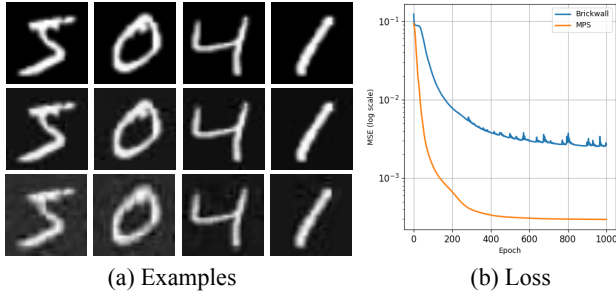


Figure 3. Results of QTN approximation to MNIST using `tneq_qc`, where (a) shows original images (first row), reconstruction from MPS (second row), and reconstruction from “brick wall” (third row); (b) shows the values of the loss function in the learning phase of QTNs in MPS and “brick wall” format.

sults. The sub-figure (a) visualizes the reconstruction from QTNs in MPS and brick wall format using the software, and (b) shows corresponding curve of the loss functions in the training phase.

5.2. Quantum tensor network equation

Up to tensor factorization, `tneq_qc` also supports of using a QTN to approximate another QTN, which is of importance, particularly in the field of quantum computing.

Suppose we have a QTN defined in the code as follows:

```
graph_teacher = QCTNHelper.mps(NUM_QUBITS,
    BOND_DIM_MPS, PHYS_DIM)
qtn_teacher = QCTN(graph_teacher, backend=
    backend).auto_init()
```

which defines a *teacher* MPO with the builtin function `QCTNHelper.mps` and randomly-generated but fixed values of core tensors. The teacher QTN is used to simulate any target quantum operator, which is complex, unitary, and modeled by QTN.

On the other side, we define a student QTN as follows:

```
graph_student = QCTNHelper.brickwall(
    NUM_QUBITS, NUM_LAYERS_BRICKWALL, PHYS_DIM)
qtn_student = QCTN(graph_student, backend=
```

```
backend).auto_init()
qtn_student.requires_grad_(True)
```

in which the graph is defined with brickwall. Note that the last line in the code allows that the core tensors are learnable.

The goal is thus to use the student QTN to approximate the teacher QTN. In the code, we set up the training as follows:

```
combined = QCTN.concat(['s', qtn_student),
    ('t', qtn_teacher.hermit())])
combined.set_trace('all')

# Training loop
opt = create_optimizer("sgdg", combined.
    parameters(), backend=backend, lr=LR)

for step in range(1, NUM_EPOCHS + 1):
    loss_val, grads = engine.
    contract_for_gradient(combined, target=
    float(PHYS_DIM ** NUM_QUBITS), loss='
    mse_tneq')
    opt.step(list(grads))
```

In the code, the first line obtains a new QTN, which models the trace between the student QTN and the Hermitian of the teacher QTN. After the trace construction, `opt` generates the used optimizer in the training phase. Unlike “Adam” used in the last experiment, here we use “sgdg”, another built-in optimizer in the software for gradient-descent optimization on Stefield manifold. The “sgdg” optimizer guarantees that the optimized student QTN preserves the unitary property.

5.3. Probabilistic modeling

We next consider density estimation with a quadratic-form QTN, which is the most basic probabilistic modeling workflow supported by `tneq_qc`. Given an input sample x , the model evaluates a probability-like score through the quadratic form $\langle \psi(x) | A^\dagger M_x A | \psi(x) \rangle$, where `CircuitState` encodes the input, MPS provides the trainable tensor network, and `MeasureMatrix` injects data-dependent measurement operators generated from a Hermite basis.

Model and training setup. In the code, this workflow is encapsulated by the `Quadratic` module, which builds a five-segment tensor network of the form $CS + MPS + Mx + MPS^\dagger + CS^\dagger$. Training is then carried out by combining `EngineCommon`, `DataGenerator`, and a gradient-based optimizer such as SGD. The main loop injects new measurement tensors into the m_x cores and minimizes the negative log-likelihood through `contract_for_gradient(..., loss='nll')`:

```
backend = BackendFactory.create_backend("
    pytorch", device="cpu", dtype="complex64")
engine = EngineCommon(backend=backend,
```



```

strategy_mode="full")
data_gen = DataGenerator(backend, mx_K=
PHYS_DIM)

model = Quadratic(nqubits=N_QUBITS,
bond_dim=BOND_DIM,
                    phys_dim=PHYS_DIM,
backend=backend).auto_init()
model._submodules["mps"].requires_grad_(
True)
combined = model.build()

optimizer = SGD(backend.parameters(),
backend, lr=LR)
data_fn = make_data_fn(data_gen, combined,
batch_size=BATCH_SIZE, K=PHYS_DIM)

for step in range(1, N_STEPS + 1):
    data_fn(step)
    loss_val, grads = engine.
contract_for_gradient(combined, target=1,
loss="nll")
optimizer.step(list(grads))
    
```

Listing 1. Quadratic-form density estimation.

Evaluation and outputs. After training, the learned MPS cores can be saved and reused for inference. The software further supports three types of post-training evaluation: marginal probability heatmaps computed on a grid, autoregressive sampling through the built-in sampler, and histogram-based KL-divergence estimation between training samples and generated samples. These utilities make the probabilistic modeling pipeline self-contained, covering model construction, differentiable training, checkpointing, and downstream visualization in a unified workflow.

5.4. Performance evaluation for distributed deployment

Finally, we evaluate the distributed execution module of `tneq_qc` using an MPS-of-Brickwall training workload implemented in `train_mps_brickwall_dist.py`. The goal of this benchmark is to quantify when distributed execution improves wall-clock performance and when it primarily serves as a memory-saving mechanism. We report the average per-step training time after removing the first two warm-up iterations, together with the peak resident memory (VmHWM), which reflects the actual physical memory usage.

Experimental setup. All experiments use a block size of 7, overlap 1, batch size 1024, `complex64` tensors, and the CPU `gloo` backend. We compare single-process execution with two-process distributed execution. In all distributed runs, the tensor network is partitioned by qubit index into two balanced parts, and the number of cross-partition edges is fixed at 4, which is the lower bound induced by the five-segment sandwich topology used in this benchmark.

Scaling with physical dimension. We first fix the number of qubits to 49 and vary the physical dimension. The results are shown in Table 1. Distributed execution provides substantial speedup when the local contraction dominates the runtime, especially for moderate physical dimensions. However, when the physical dimension becomes large, the cost of reduction and communication catches up with the local compute cost, so the speedup vanishes while the memory benefit remains significant.

Scaling with qubit count. Next, we fix the physical dimension to 8 and increase the number of qubits. As shown in Table 2, the two-process configuration consistently achieves about $3.6\text{--}4.1\times$ speedup from 49 to 145 qubits, while also reducing memory usage by 19% to 30%. This trend is expected because, under the chosen partition, the number of cross-partition edges stays constant as the network grows, so the communication overhead does not increase proportionally with the problem size.

Large-scale memory behavior. Finally, Table 3 compares physical dimensions 8 and 16 at 145 qubits. The larger configuration shows the main systems-level benefit of distributed execution: even when speedup is limited, memory usage is reduced substantially. For example, at 145 qubits and physical dimension 16, the peak RSS drops from 17.1 GB to 10.2 GB, corresponding to roughly 6.9 GB of memory saved per run. This result is consistent with the fact that intermediate tensor size grows exponentially with the number of open edges, making distributed execution increasingly valuable as a memory-management mechanism.

Discussion. Overall, the benchmark reveals two clear regimes. When the physical dimension is moderate (up to about 8 in our setting), distributed execution yields substantial speedup because local contraction dominates the runtime. When the physical dimension becomes large, the reduction stage becomes the main bottleneck: with four unavoidable cross-partition edges, the communication volume scales as $\text{batch} \times \text{phys_dim}^4$, which is already about 512 MB per reduction at physical dimension 16 and batch size 1024 under `complex64`. Consequently, the distributed implementation should be viewed as both a performance optimization for moderate workloads and a memory-enabling mechanism for larger ones. This behavior is important in practice, since many large tensor-network models are constrained by memory before they are constrained by raw floating-point throughput.

6. Related Libraries

The ecosystem of tensor-network (TN) and quantum simulation software is extensive, spanning general-purpose TN frameworks, tensor decomposition libraries, and specialized

Table 1. Distributed benchmark with 49 qubits and varying physical dimension. Entries report avg step time / Peak RSS.

phys_dim	standalone	2 proc	speedup	memory saving
2	0.54 s / 521 MB	0.31 s / 512 MB	1.7×	-2%
4	12.10 s / 712 MB	0.55 s / 703 MB	22.0×	-1%
8	17.91 s / 2173 MB	5.04 s / 1756 MB	3.6×	-19%
12	28.52 s / 5442 MB	24.13 s / 4529 MB	1.2×	-17%
16	73.87 s / 9140 MB	80.45 s / 6178 MB	0.9×	-32%

Table 2. Distributed benchmark with fixed physical dimension 8 and varying qubit count. Entries report avg step time / Peak RSS.

qubits	standalone	2 proc	speedup	memory saving
49	17.91 s / 2173 MB	5.04 s / 1756 MB	3.6×	-19%
97	38.34 s / 3229 MB	9.32 s / 2348 MB	4.1×	-27%
145	61.01 s / 4531 MB	16.64 s / 3157 MB	3.7×	-30%
241	120.66 s / 7045 MB	not tested	—	—

utilities for contraction optimization and high-performance execution. These tools have significantly advanced both scientific computing and machine learning applications.

Tensor-network and simulation libraries. General TN libraries such as *TensorNetwork*, *Quimb*, and *TeNPy* provide flexible abstractions for constructing and contracting tensor networks, often with support for multiple numerical backends and optimized contraction paths. They are widely used in physics-inspired simulations, including matrix product states (MPS), projected entangled pair states (PEPS), and quantum dynamics algorithms such as DMRG and TEBD. Other tools such as *PyTreeNet* and *TensorCircuit* further extend these capabilities to hierarchical tensor networks and large-scale quantum circuit simulation. While these libraries offer strong support for numerical simulation and domain-specific algorithms, they typically focus either on physics-oriented workflows or circuit execution, with limited emphasis on unified learning pipelines or distributed differentiable computation.

Tensor decomposition and low-rank libraries. A complementary line of work focuses on low-rank tensor representations, including CP, Tucker, and Tensor-Train (TT/MPS) formats. Libraries such as *TensorLy*, *TNTorch*, and *T3F* integrate tensor decompositions with modern machine learning frameworks, enabling differentiable tensorized models and parameter-efficient learning. Other packages, including *Scikit-TT*, *TTpy*, and *mpnum*, provide efficient implementations of TT-based numerical algorithms, while *pyDNTNK* and *Tensap* address large-scale or functional approximation settings. These libraries are effective for tensor factorization and compression, but they generally abstract away explicit network topology and contraction strategy design, which are central to tensor-network-based modeling.

Specialized tools and high-performance backends. Several utilities target specific bottlenecks in TN computation. For example, *opt_einsum* provides optimized contraction

path planning across multiple array backends, *TensorGrad* enables symbolic manipulation and differentiation of tensor expressions, and *cuTensorNet* delivers highly optimized GPU kernels for large-scale TN contraction. These tools are often integrated into broader pipelines to improve efficiency, but they typically operate at a lower level of abstraction.

Position of `tneq_qc`. In contrast to the above libraries, `tneq_qc` is designed as a lightweight and modular research framework that bridges tensor-network modeling, learning, and distributed execution. It differs from existing approaches in three key aspects. First, it introduces a compact ASCII-based graph representation, enabling rapid and human-readable specification of quantum-circuit-like tensor networks. Second, it adopts a strategy–execution separation, allowing contraction strategies to be developed and tested independently of backend implementations. Third, it natively supports differentiable computation and distributed execution within a unified interface, facilitating end-to-end learning workflows on large-scale TN models.

Overall, `tneq_qc` is complementary to existing libraries: rather than replacing specialized simulation or decomposition tools, it provides a flexible “middle layer” for experimenting with tensor-network structures, contraction strategies, and learning-based formulations in a unified and extensible manner.

7. Discussion and Conclusion

Summary. This paper presented `tneq_qc`, a lightweight software framework for building, composing, contracting, and training quantum tensor networks. The framework unifies graph-based model specification, scaled tensor representations, strategy-driven contraction, multi-backend execution, and distributed deployment within a single programming interface. Through examples including tensor factorization, QTN-to-QTN approximation, density estima-

Table 3. Comparison at 145 qubits for physical dimensions 8 and 16. Entries report avg step time / Peak RSS.

phys_dim	standalone	2 proc	speedup	memory saving
8	61.01 s / 4531 MB	16.64 s / 3157 MB	3.7×	-30%
16	234.80 s / 17114 MB	251.17 s / 10205 MB	0.9×	-40%

tion, and distributed benchmarks, we showed that the same software stack can support both rapid prototyping and executable end-to-end learning workflows.

Practical implications. From a software perspective, the main contribution of `tneq_qc` is the separation of concerns between topology specification, contraction planning, backend execution, and optimization. This design makes it possible to reuse the same tensor-network model under different strategies and hardware settings, while keeping the application-level code concise. Empirically, the results in this paper suggest that the framework is effective not only for expressing quantum-inspired models, but also for exposing useful performance trade-offs between standalone and distributed execution.

Limitations. Several parts of the current implementation remain research-oriented. The strategy selection logic is still heuristic in some cases, the set of supported contraction strategies is limited, and some utility components are tailored toward experimentation rather than production robustness. In addition, although the distributed engine provides clear memory benefits and strong speedups in favorable regimes, communication overhead becomes a bottleneck for larger physical dimensions.

Future work. There are several directions for extending the framework. On the systems side, an immediate next step is to improve cost modeling and introduce more structure-aware or hardware-aware contraction strategies. On the modeling side, it would be useful to broaden the collection of built-in modules and probabilistic learning pipelines. On the engineering side, additional backend support, richer checkpointing and profiling tools, and more comprehensive large-scale benchmarks would further strengthen the usability of the framework.

Acknowledgements

This work is partially supported by the collaborative research program 2025 information initiative center, Hokkaido university, Sapporo, Japan, and FY2025 Google Academic Grants J250002585.